Why mod_perl 2.0 Sucks

Why mod_perl 2.0 Rocks

Geoffrey Young

geoff@modperlcookbook.org

A Long Time Ago...

- I figured it was time to take mod_perl 2.0 for a test drive...

Laziness, Impatience, Hubris

- "I know mod_perl pretty well..."
- "Compat layers are for weenies..."
- "It's still HTTP, so not much can change..."
- "This will be easy."

The Adventure Begins

- So, I take an existing, complex program and just plug it in...

Let's Try Again

- "Maybe something simpler won't dump core."
- So, I try a new but simple program...

```
package My::Foo;

use Apache::Constants qw(OK);

use strict;

sub handler {
  my $r = shift;
  my $log = $r->server->log;
  $log->info('*** processing uri ', $r->uri);
  my $foo = $r->dir_config->get('Foo');
  $r->content_type('text/plain');
  $r->print("found $foo");
  return OK;
}

1;
```

```
package My::Foo;

use Apache::Const qw(OK);
use Apache::RequestRec ();
use Apache::Server ();
use Apache::Log ();
use Apache::RequestUtil ();
use APR::Table ();
use Apache::RequestIO ();

use strict;

sub handler {

  my $r = shift;

  my $log = $r->server->log;

  $log->info('*** processing uri ', $r->uri);

  my $foo = $r->dir_config->get('Foo');

  $r->content_type('text/plain');

  $r->print("found $foo");

  return OK;
}

1;
```

Gripes

- Different
- un-DWIMmy
- Examples and docs difficult to find
- Too many calls to `grep(1)`
- Incomplete

DWIMmy

```
use Apache::Constants qw(OK);
```

un-DWIMmy

```
use Apache::Const qw(OK);
use Apache::RequestRec ();
use Apache::Server ();
use Apache::Log ();
use Apache::RequestUtil ();
use APR::Table ();
use Apache::RequestIO ();
```

Apache::Log

- 1.0 required `use()`ing `Apache::Log`

```
use Apache::Constants qw(OK);
use Apache::Log ();

my $log = $r->server->log();
```

- Not really all that different...

Something Better Here?

```
use Apache::Const qw(OK);
use Apache::RequestRec ();
use Apache::Server ();
use Apache::Log ();
use Apache::RequestUtil ();
use APR::Table ();
use Apache::RequestIO ();
```

Apache::RequestRec

- Access to `request_rec` structure only
- Makes `$r` more like `r`
- Reinforces what mod_perl really is
  - access to the Apache C API in Perl

Apache::RequestRec

- New methods that mirror their C `request_rec` counterparts
  - `$r->notes()`
  - `$r->content_type()`
  - `$r->headers_out()`
  - `$r->per_dir_config()`
  - `$r->pool()`

This ROCKS!

- Yes, mod_perl 2.0 has an `assbackwards()` method
- Access to `assbackwards` slot in the `request_rec`
- Saying `assbackwards` rocks
- Indicative of something even cooler...

Access to *Everything*

- The goal of mod_perl has always been complete access to the Apache C API
- 1.0 - API partially opened over time
- 2.0 - entire API is open by default
  - autogenerated accessors/mutators
  - only closed where a Perl API isn't appropriate
- Complete access rocks

APR::Pool

- Access to Apache pools (finally)
- Old features, new syntax

```
$r->pool->cleanup_register($cv);
```

Handy New Features

- Per-connection cleanups

```
$r->connection->cleanup_register($cv);
```

- Reconfiguration cleanups

```
$r->process->pconf->cleanup_register($cv);
```

- Pool management

```
{
    my $pool = APR::Pool->new();

    $pool->cleanup_register($cv);
}

# $cv executes when $pool is DESTROY()ed
```

Output Filters

- mod_perl has been able to filter content for years
  - limited to mod_perl generated content
    - mod_perl can do lots, like CGI and SSI
- Output filters let you filter *everything*
  - no matter who generates the content

Perl Output Filters

- Complete access to the Apache API
  - raw bucket brigade manipulations
- mod_perl also makes Apache Perlish
  - streaming filter API
- Streaming filters are insanely easy to write

Streaming Filter API

```
package My::Filter;

use Apache::Filter ();

use Apache::Const qw(OK);

sub handler {

  my $f   = shift;

  while ($f->read(my $buffer, 1024)) {

    # do something with $buffer

    $f->print($buffer);
  }

  return OK;
}

1;
```

httpd.conf

```
PerlOutputFilterHandler My::Filter
```

mod_perl as Glue

- mod_perl's main goal is to provide a Perl version of the Apache C API
- Secondary to that, mod_perl provides glue between Perl and Apache
  - access the Apache API via XS
- mod_perl 2.0's internals are cleaner
  - easier to access (yet) unsupported Apache features in Perl

Bigger Community

- 1.0 had one (brilliant) primary developer
- 2.0 is more community based
  - more core developers
  - more interested parties
    - httpd-test
    - apreq
    - p5p
  - more platforms
    - far better win32 support
- Many eyes approach makes for better software

Apache-Test

- Single best part of mod_perl 2.0
- Framework for unit testing Apache-based applications
  - mod_perl handlers
  - CGI scripts
  - SOAP servers
  - more
- So useful, it's now part of the `httpd-test` project

Well Tested

- `Apache-Test` used in code development from the start
- More, better tests in 2.0
- Comes very close to exercising every part of the external API
- Makes for better software

Why mod_perl 2.0 rocks

- More features
- Better software product

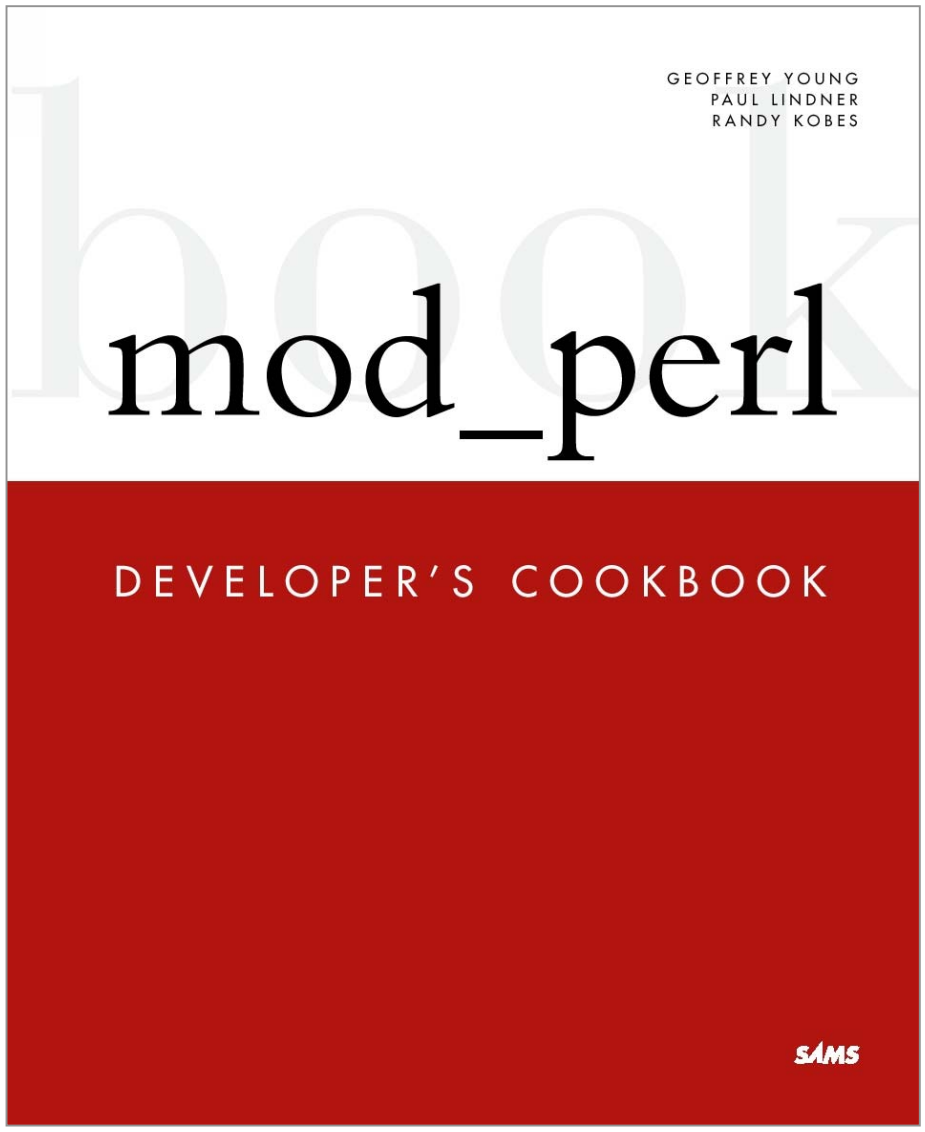
Slides

- These slides freely available at some long URL you will never remember…

`http://www.modperlcookbook.org/~geoff/slides/ApacheCon/2003/`

- Linked to from my homepage

`http://www.modperlcookbook.org/~geoff`



http://www.modperlcookbook.org/

This presenation has ended.
Click anywhere to return.

_

